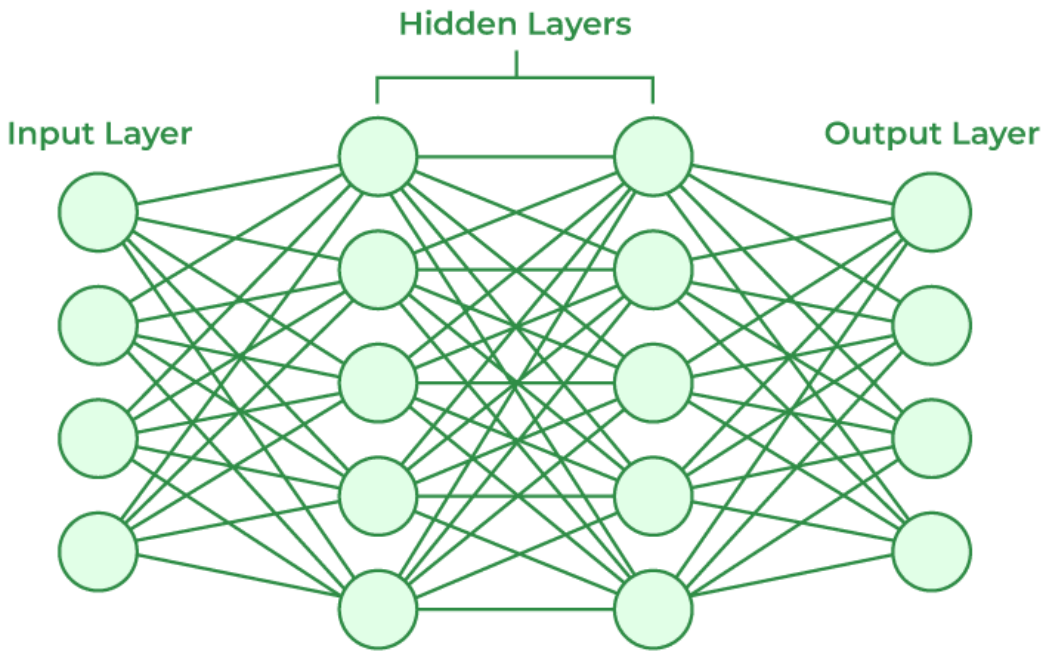
Artificial Neural Network

ANN

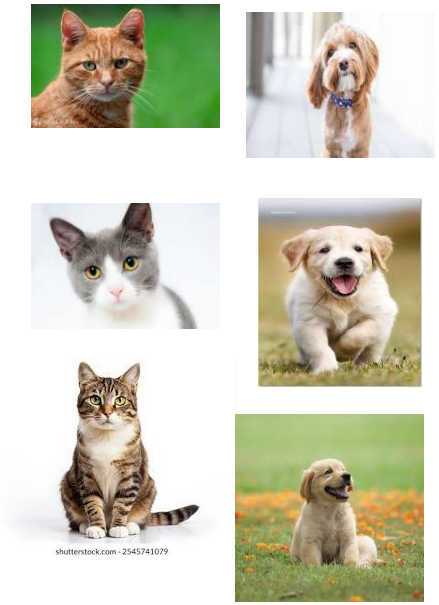
الشبكة العصبية الاصطناعية.

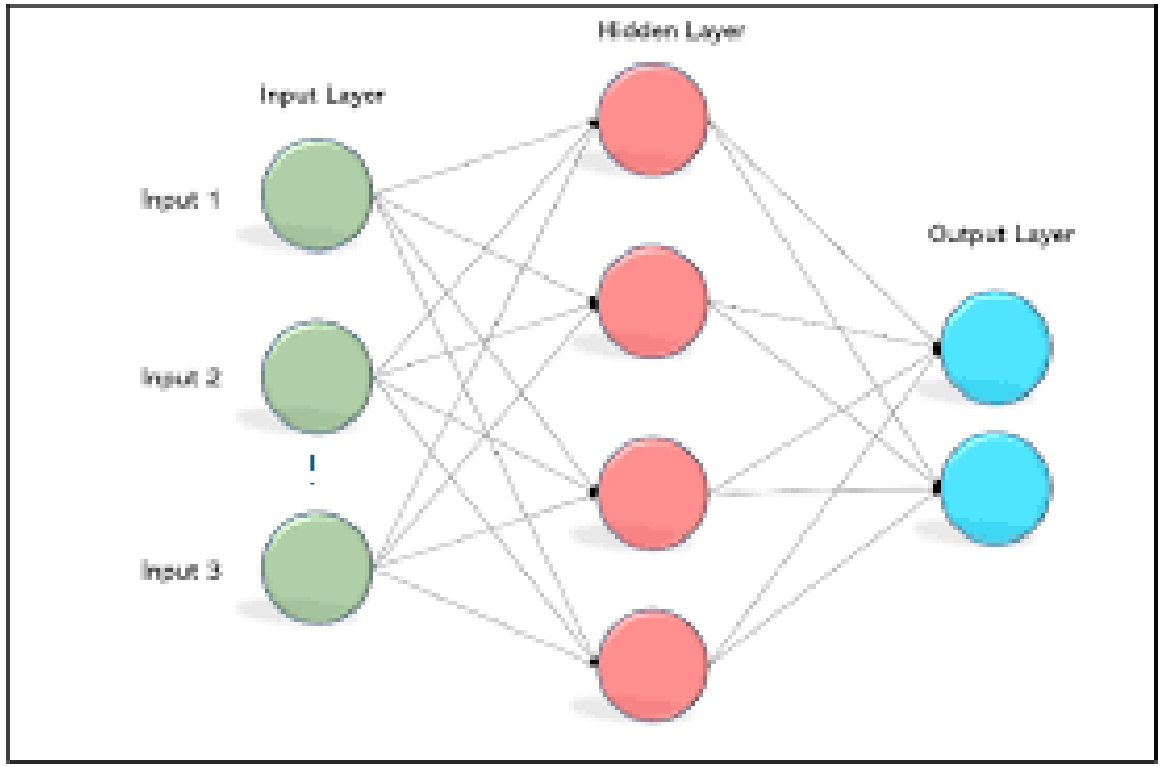
Artificial Neural Network (ANN)

Artificial Neural Networks (ANNs) were widely used in the 1980s and 1990s, aiming to mimic how the human brain processes information. Just like the brain uses neurons to process data and make decisions, ANNs use artificial neurons to analyze data, identify patterns and make predictions. These networks consist of layers of interconnected neurons that work together to solve complex problems. The key idea is that ANNs can "**learn**" from the data they process, just as our brain learns from experience.



Cat (+1) Dog (-1)





Cat (+1)

Dog (-1)

Key Components of an ANN

•**Input Layer:** This is where the network receives information. For example, in an image recognition task, the input could be an image.

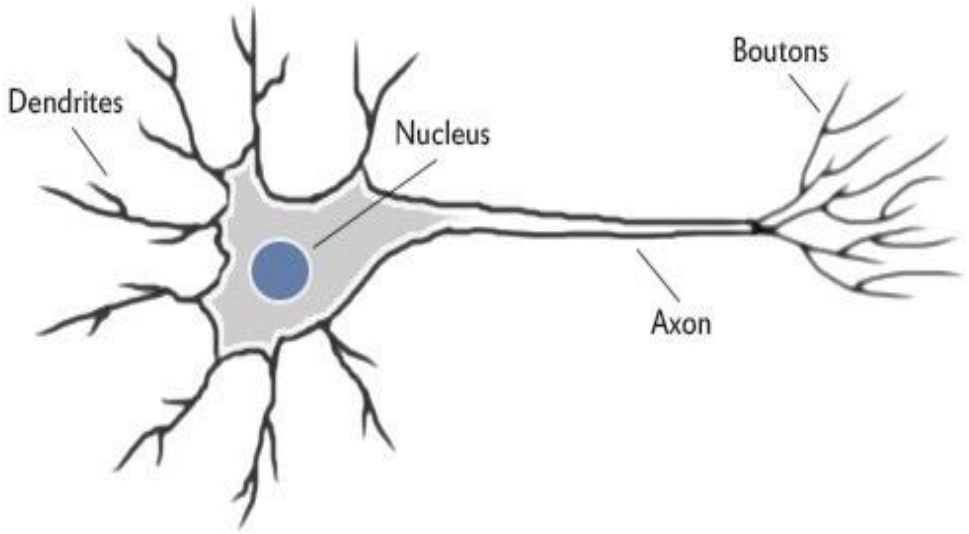
•**Hidden Layers:** These layers process the data received from the input layer. The more hidden layers there are, the more complex patterns the network can learn and understand.

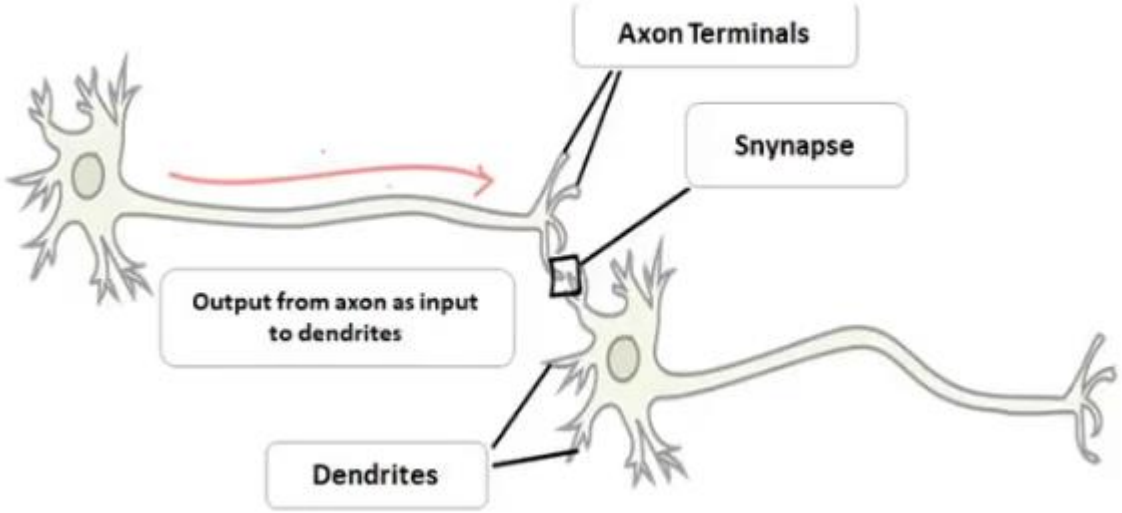
•**Output Layer:** This is where the final decision or prediction is made. For example, after processing an image, the output layer might decide whether it’s a cat or a dog.

Neural Network

Introduction:

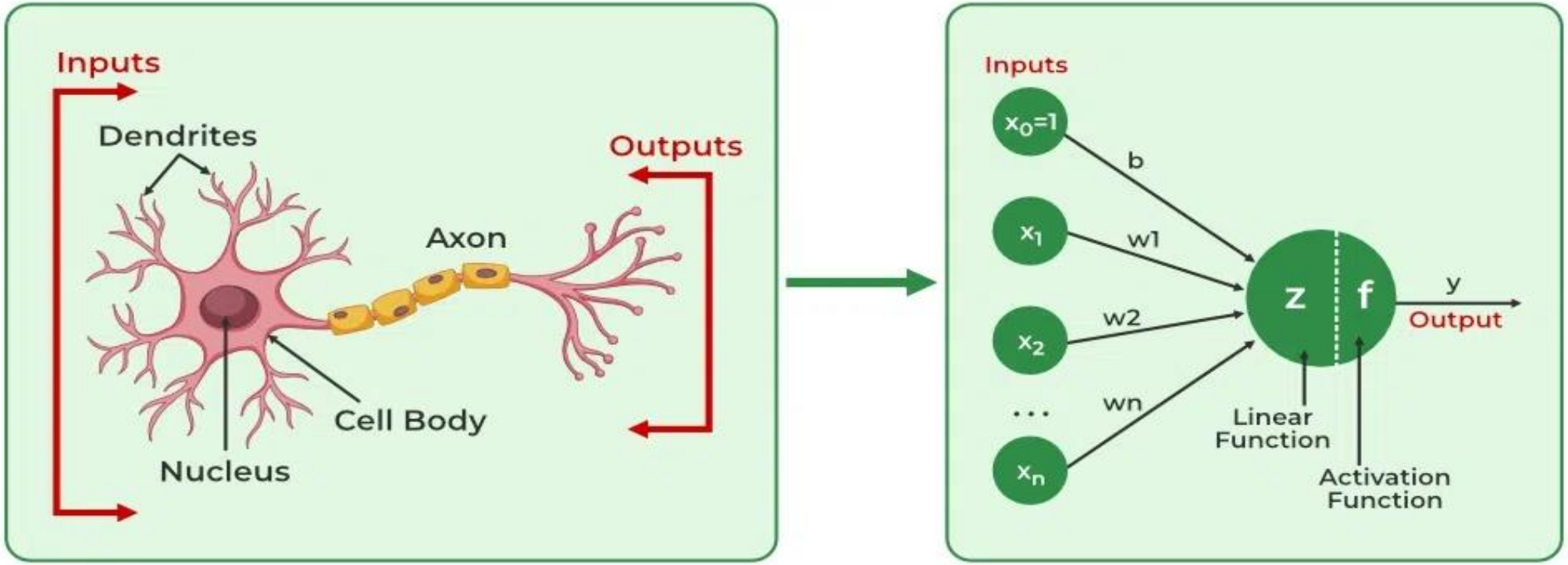
Our brain uses a vast interconnected network of neurons for information processing and modelling the world around us. A neuron is a cell capable of transmitting information to other neurons through its various connections (synapses).





Neural connections in the human brain

Biological Neural **Artificial Neural**

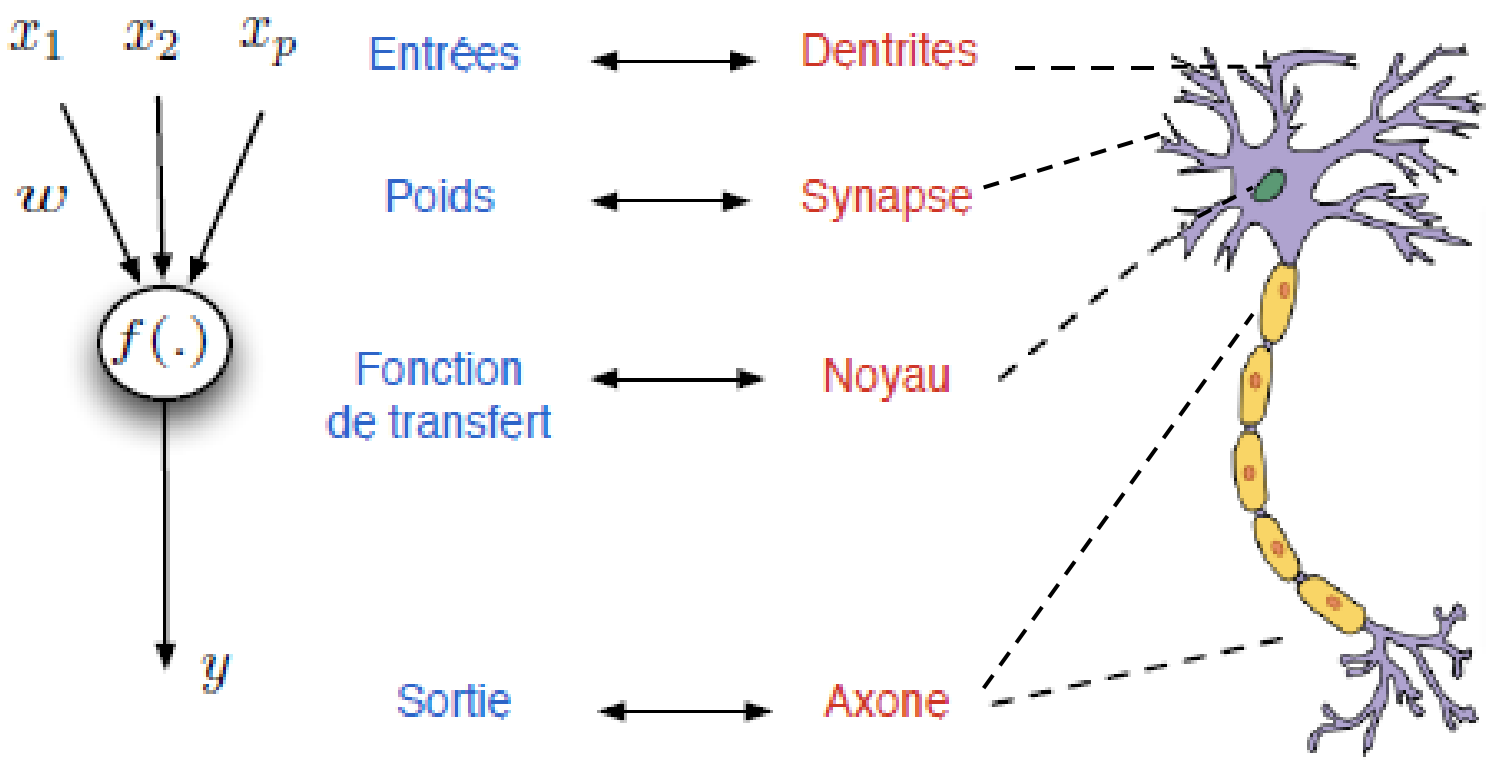


- Collects inputs from other neurons using dendrites. Dendrites are used for receiving input messages from other cells

- Sums all the inputs and fires if the value is greater than a threshold.

- The fired signal is then sent to other neurons through the Axon. Axon conducts signals from the cell body to the output terminals

A single neuron:



Artificial Neuron **Biological Neuron**

What is a Perceptron?

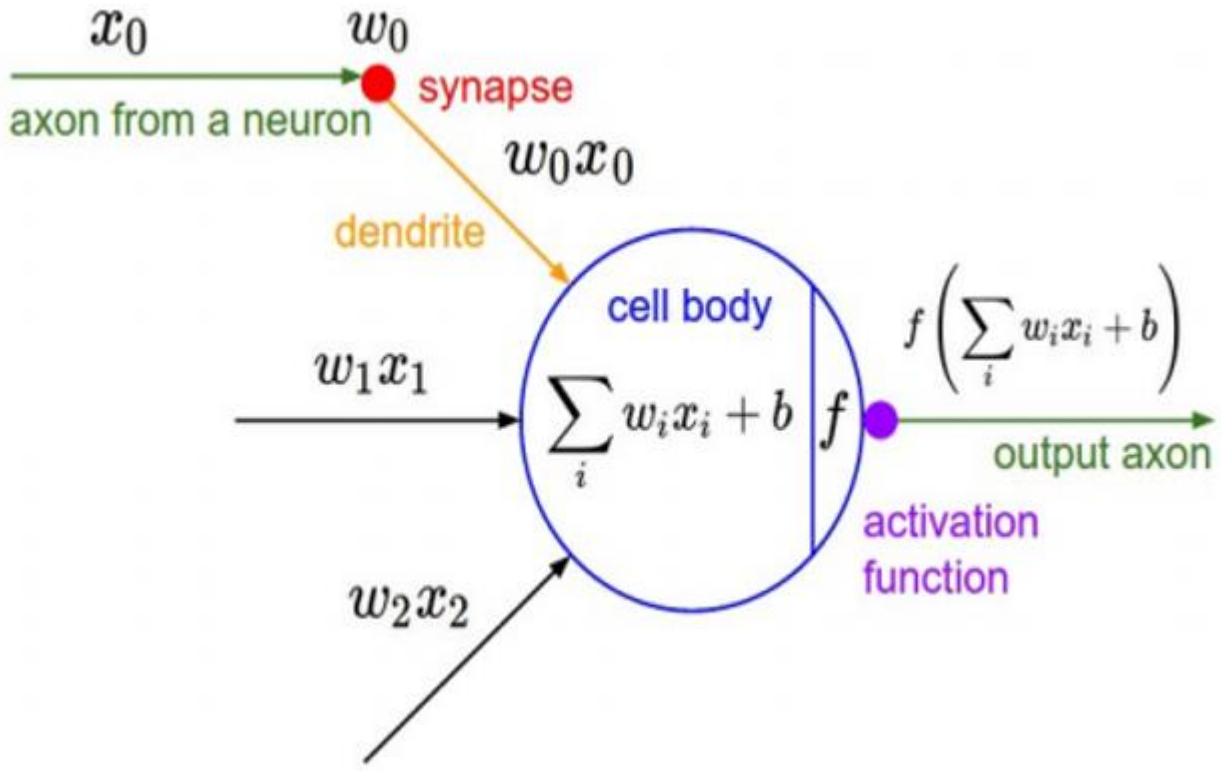
A perceptron is a type of neural network that performs binary classification that maps input features to an output decision, usually classifying data into one of two categories, such as 0 or 1.

A perceptron consists of a single layer of input nodes that are fully connected to a layer of output nodes. It is particularly good at learning **linearly separable patterns**.

Perceptron

Mathematically, we can represent a neuron as follows (introduced by **McCulloch and Walter Pitts** in the 1940).

McCulloch-Pitts neuron Model



This foundational model has played a crucial role in the development of more advanced neural networks and machine learning algorithms.

Basic Components of Perceptron

A Perceptron is composed of key components that work together to process information and make predictions.

•**Input Features:** The perceptron takes multiple input features, each representing a characteristic of the input data.

•**Weights:** Each input feature is assigned a weight that determines its influence on the output. These weights are adjusted during training to find the optimal values.

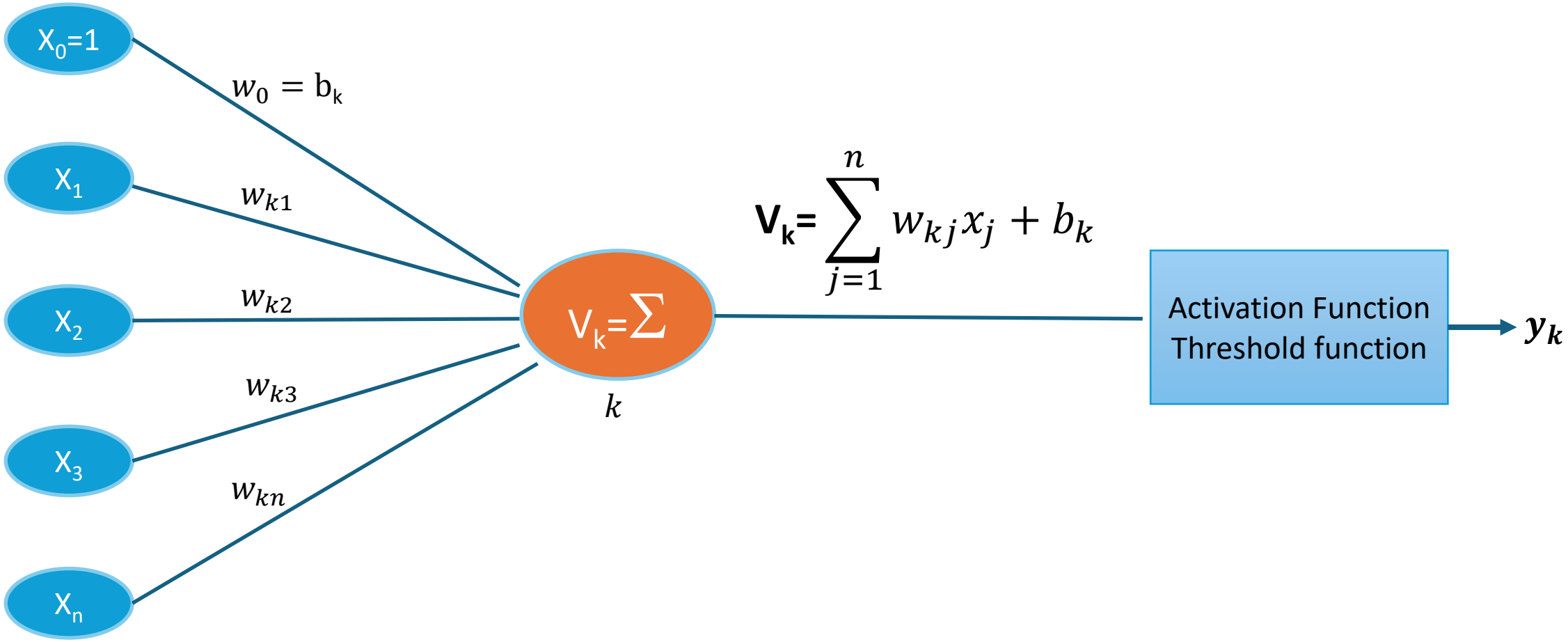
•**Summation Function:** The perceptron calculates the weighted sum of its inputs, combining them with their respective weights.

•**Bias:** The bias term helps the perceptron make adjustments independent of the input, improving its flexibility in learning.

•**Activation Function:** The weighted sum is passed through the **Heaviside step function**, comparing it to a threshold to produce a binary output (0 or 1).

•**Output:** The final output is determined by the activation function, often used for **binary classification** tasks.

•**Learning Algorithm:** The perceptron **adjusts its weights and bias** using a learning algorithm, such as the **Perceptron Learning Rule**, to **minimize prediction errors**.



- **Model of an artificial neuron**

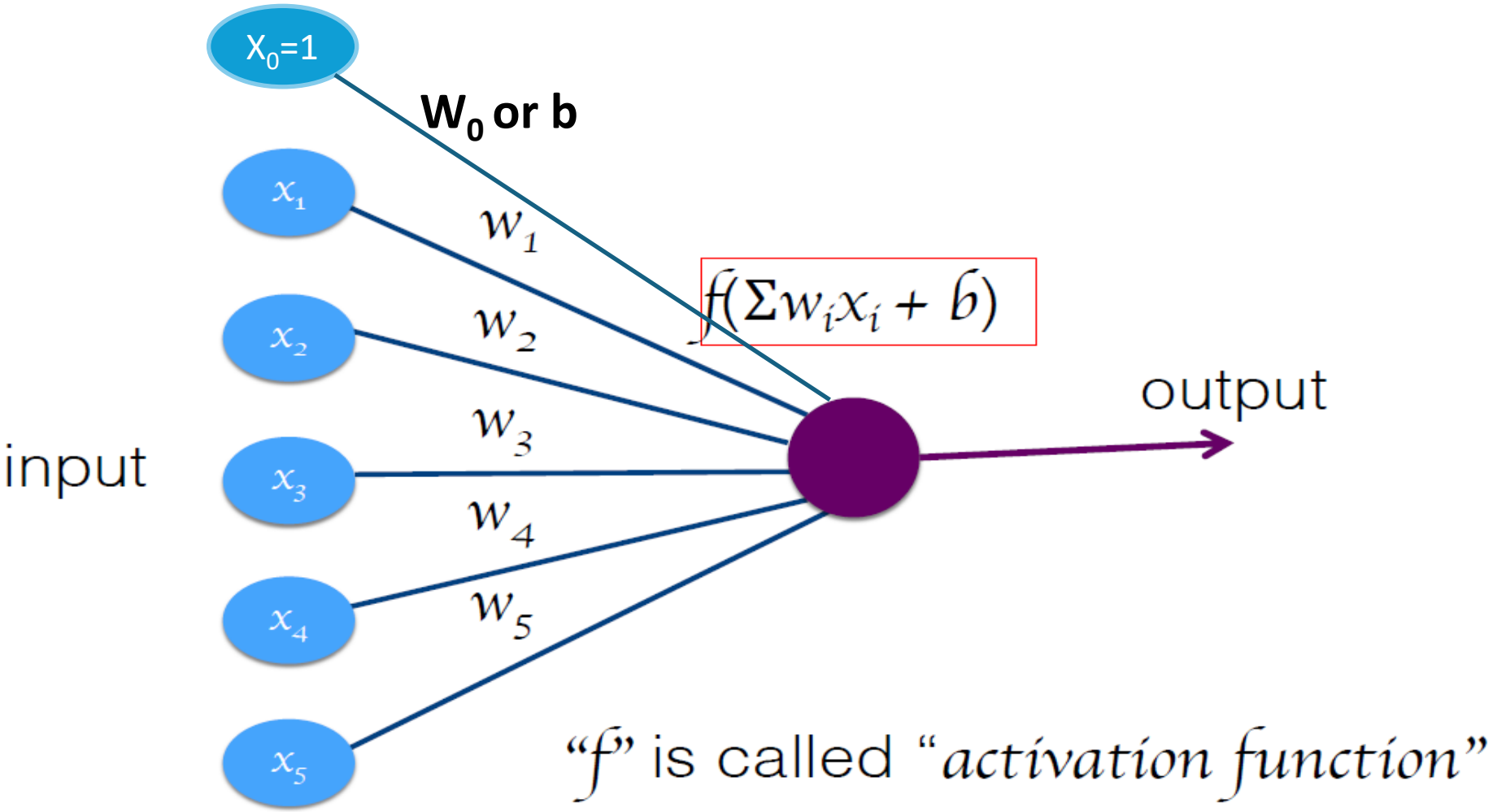
Types of Perceptron

1.Single-Layer Perceptron SLP is a type of perceptron that is limited to learning linearly separable patterns. It is effective for tasks where the data can be divided into distinct categories through a straight line. While powerful in its simplicity, it struggles with more complex problems where the relationship between inputs and outputs is non-linear. For example, a single perceptron can handle simple binary classification

2.Multi-Layer Perceptron MLP possess enhanced processing capabilities as they consist of two or more layers, adept at handling more complex patterns and relationships within the data. For example, a complex tasks require multiple perceptrons organized into layers, forming a neural network.

- Model of an artificial neuron

Artificial Neuron



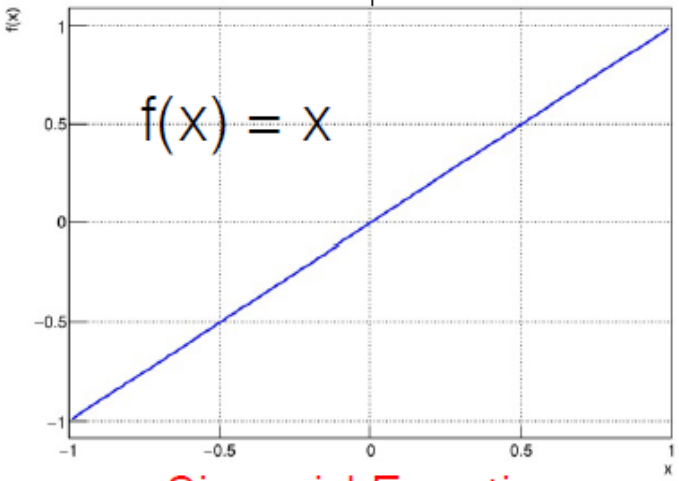
"f" is called "*activation function*"

b is a bias term → Can be represented by a node with input "1".

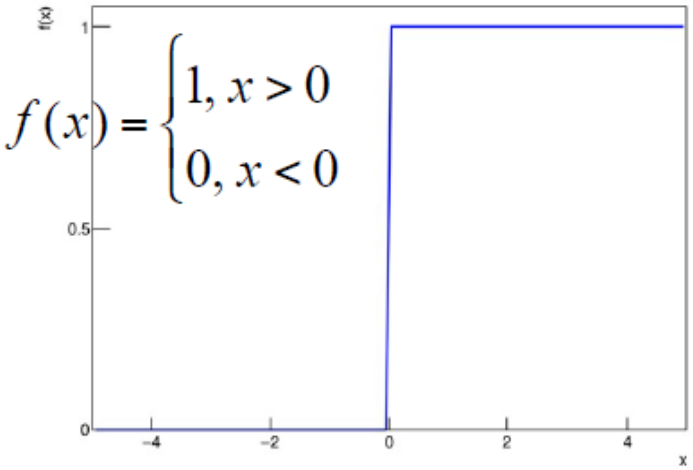
Activation Functions

It determines at what threshold the neuron will fire, OR the frequency at which a neuron fires.

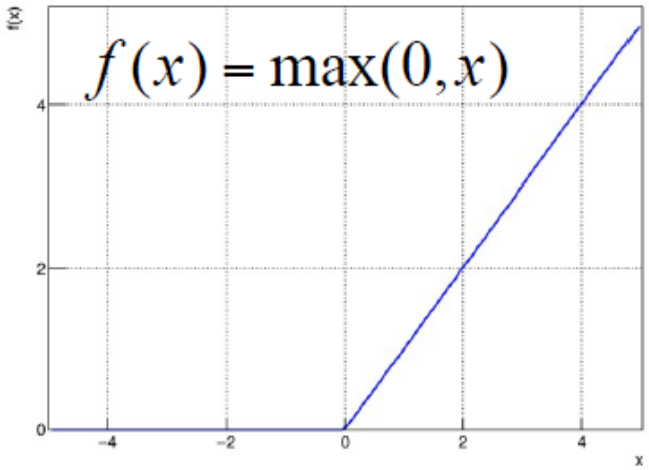
Linear Function



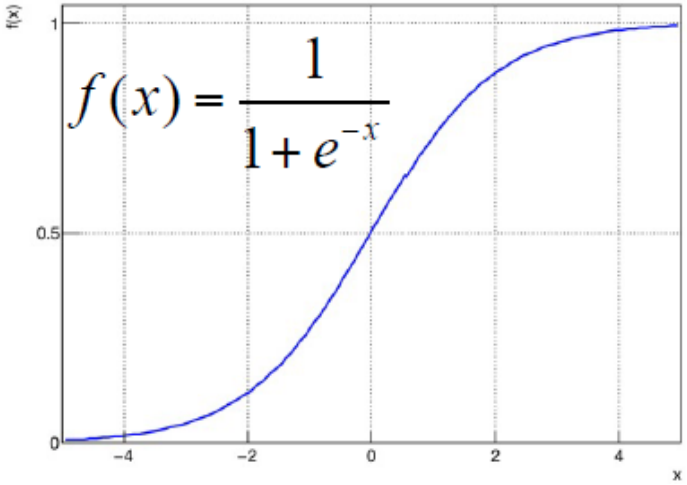
Step Function



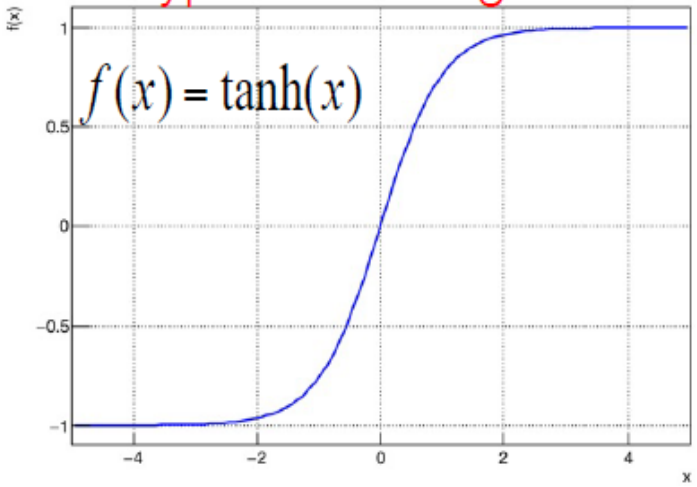
Rectified Linear Units (ReLU)



Sigmoid Function



Hyperbolic Tangent

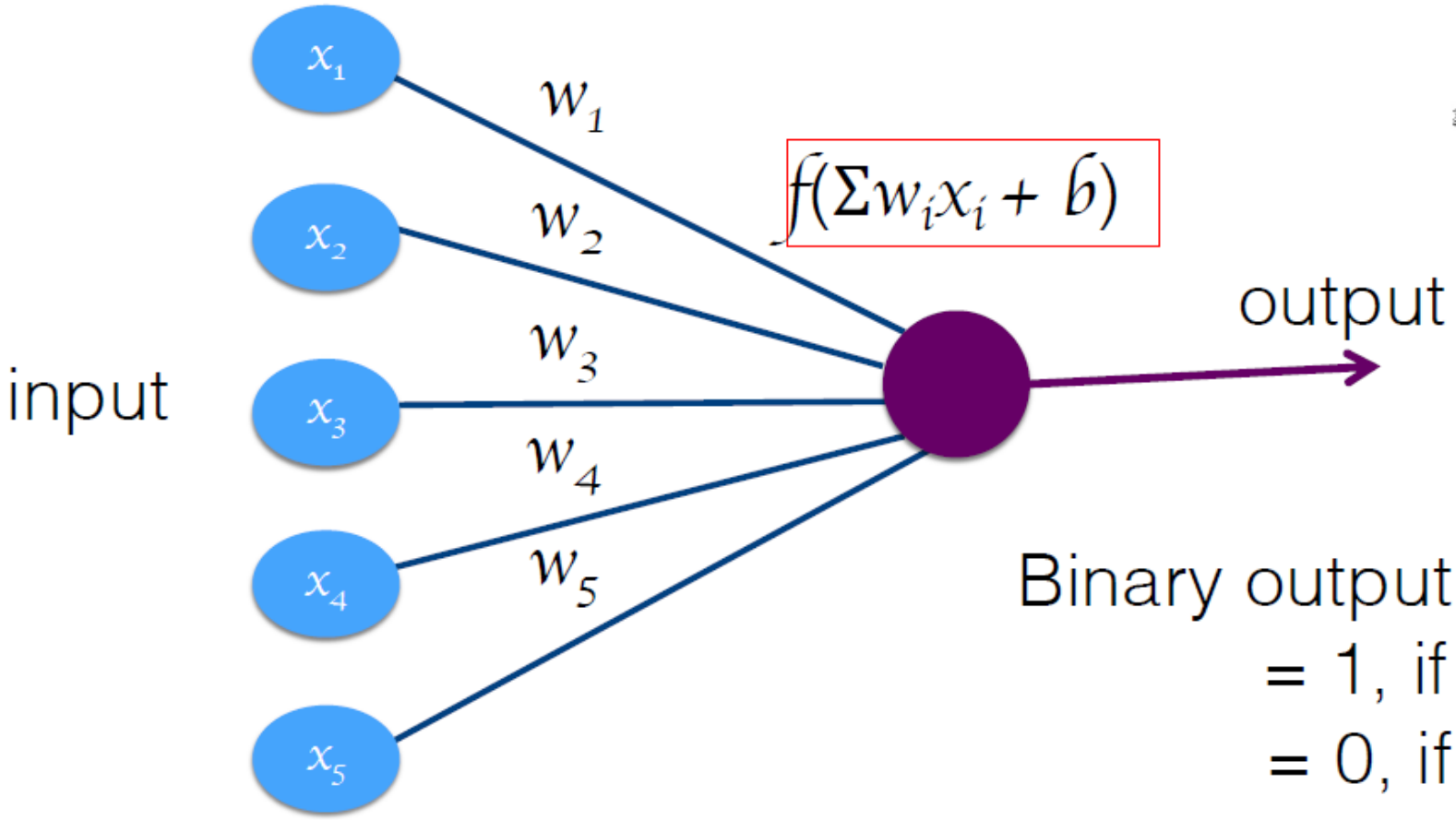


Softmax

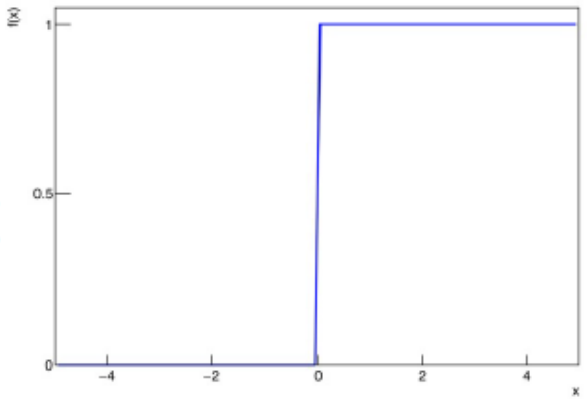
$$f(x_i) = \frac{e^{x_i}}{\sum e^{x_i}}$$

(used in output layer of a multiclassification network)

Single-Layer Perceptron



Binary Step function



Binary output:

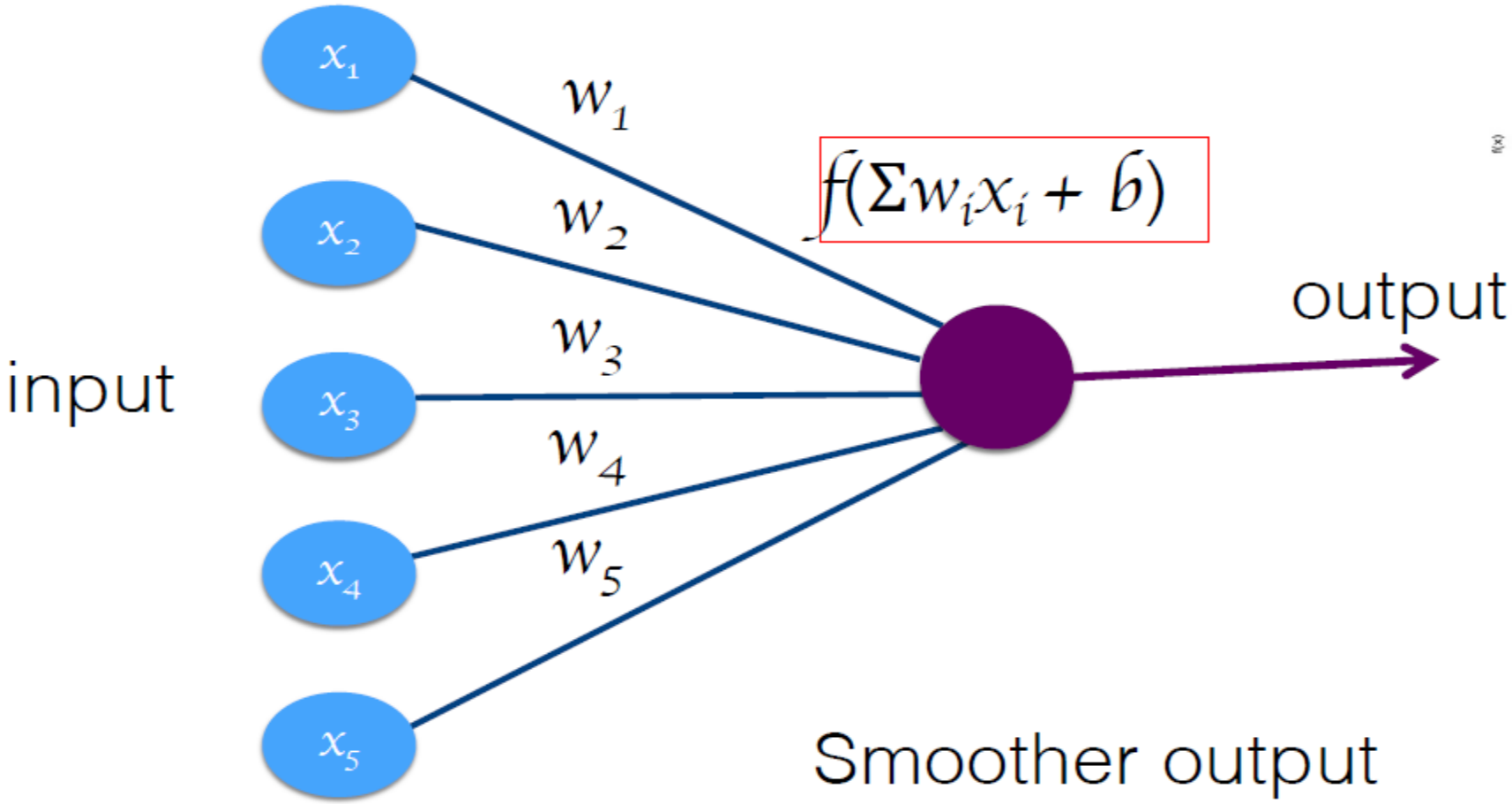
$= 1$, if $\Sigma w_i x_i + b > 0$

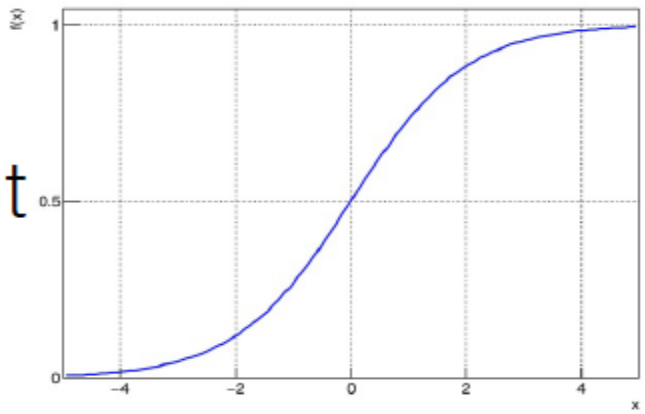
$= 0$, if $\Sigma w_i x_i + b < 0$

Perceptron

Sigmoid Neuron

$$f(x) = \frac{1}{1+e^{-x}}$$





Smoother output

Small change in weight => small corresponding change in output

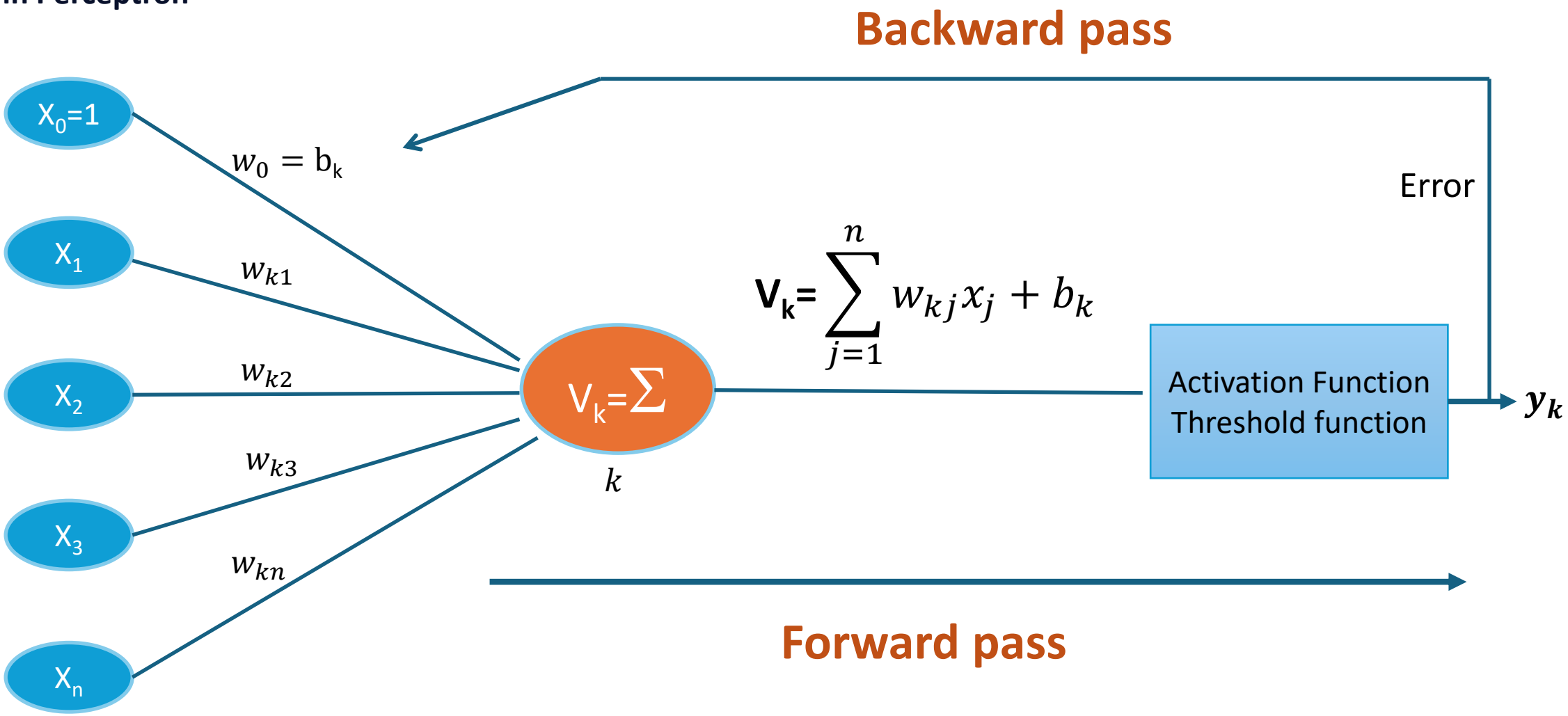
This property makes the learning possible

Build and Train Single Layer Perceptron Model

A Single Layer Perceptron is the simplest type of neural network. It consists of a single layer of neurons connected to inputs.
The perceptron model works by applying a linear function to the input data (a weighted sum) followed by an activation function (e.g., a step or sigmoid function).

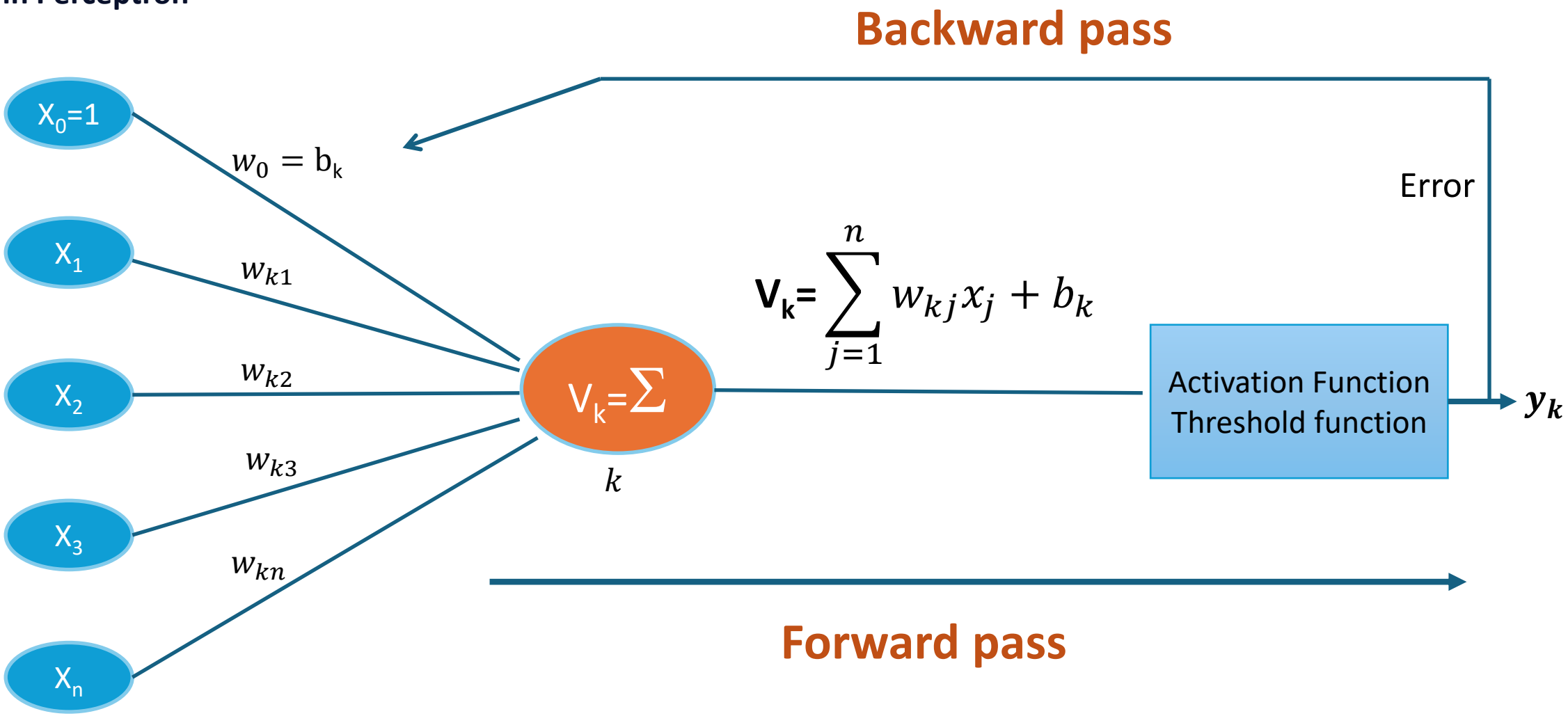
1. Initialize Parameters: Set weights and bias.
2. Forward Pass: Compute the weighted sum of inputs.
3. Activation: Use a step function to determine the output.
4. Training: Adjust weights using the Perceptron Learning Rule based on errors.

Learning in Perceptron



The whole point of training the perceptron is to adjust the weights of each input according to the training data.

Learning in Perceptron



Building and Training Single Layer Perceptron Model

1. Initialization: Start by setting the weights and bias to small random values. These weights determine the importance of each input feature.

2. Input: Feed an input data point (a vector of features) into the perceptron.

3.Weighted Sum: Calculate the weighted sum of the input features plus the bias. This is done by multiplying each input feature by its corresponding weight and adding them all up, along with the bias. weighted sum
= (w1∗x1)+(w2∗x2)+…+(wn∗xn)+b($w1*x1$)+($w2*x2$)+…+($wn*xn$)+b

4. Activation Function: Apply the activation function to the weighted sum to get the output. In a simple perceptron, this is usually a step function that outputs 1 if the weighted sum is positive, and -1 (or 0) if it's negative.

5. Prediction: Compare the perceptron's output (prediction) to the actual label of the input data point (either 1 or -1).

6. Update Weights: If the prediction is incorrect, adjust the weights and bias. The weights are updated to reduce the error, moving the decision boundary closer to correctly classifying the input. The update rule is:

$$w_i = w_i + \alpha(y - y')\, x_i$$

- α is the learning rate parameter (0 <α ≤1).
- y is the actual label
- y' is the predicted label

7. Repeat: Repeat steps 2-6 for all data points in the training set. This process is called an **Epoch**. Multiple epochs are performed until the perceptron correctly classifies all training data points or a maximum number of epochs is reached.

8. Convergence: If the data is linearly separable, the algorithm will eventually find the correct weights and bias to perfectly classify the training data. This means the perceptron has "converged."

Step 1: Initialize the weight and learning rate

The initial weights:

$w_1 = -0{,}2 \quad w_2 = 0{,}1 \quad w_0 = -0{,}2$

The learning rate: $\alpha = 0{,}1$

Input Data	x1	x2	Label
Object 1	1	1	+1
2	-1	1	-1
3	-1	-1	-1
4	1	-1	-1

Step 2. Input Data:

Object 1 : x1 =1 , x2 =1 and y= +1

Step 3. Weighted Sum:

$$\mathbf{V=} \sum_{j=1}^{n=2} w_j x_j + w_0 = w_1 x_1 + w_2 x_2 + w_0 = -0{,}2\,.1 + 0{,}1\,.1 - 0{,}2 = -0{,}3$$

Step 4. Activation Function

Binary Step function

$$f(V) = \begin{cases} 1 \; if & V > 0 \\ -1 \; if & V < 0 \end{cases}$$

So, $f(V) = -1$

5. Prediction

V < 0, so the prediction label $y' = f(V)\ is\ negative\ (-1)$, , and we know that the Actual label is positive (+1)

Prediction Error = y- y'=1-(-1)=2

6. Update Weights

Since, the prediction is incorrect, we are updating the weights (w_0, w_1, w_2) by: $w_i = w_i + \alpha(y - y')\, x_i$

$w_0 = w_0 + 0{,}1\big(1 - (-1)\big)x_0$= -0,2 + 0,2.1 = 0
$w_1 = w_1 + 0{,}1\big(1 - (-1)\big)x_1$= 0,1 + 0,2.1 = 0,3
$w_2 = w_2 + 0{,}1\big(1 - (-1)\big)x_2$= -0,2 + 0,2.1 = 0

- α is the learning rate parameter α=0,1
- y is the actual label (y =label =+1)
- y' is the predicted label (y' =$f(V_1)$=-1)

7. Repeat steps 2-6 for all object in the training set: One **Epoch**

Object 2

The initial weights:

$w_1 = 0{,}3$ $w_2 = 0$ $w_0 = 0$

The learning rate: $\alpha = 0{,}1$

Input Data	x1	x2	Label
1	1	1	+1
Object 2	-1	1	-1
3	-1	-1	-1
4	1	-1	-1

$$\mathbf{V=}\sum_{j=1}^{n=2} w_j x_j + w_0 = w_1 x_1 + w_2 x_2 + w_0 = 0{,}3\,.\,(-1) + 0\,.\,1 - 0 = -0{,}3$$

The prediction value y' $= f(V_1) = -1$The prediction is correct, y = y', it's Okay.

Object 3

The initial weights:

$w_1 = 0{,}3 \quad w_2 = 0 \quad w_0 = 0$

The learning rate: $\alpha = 0{,}1$

Input Data	x1	x2	Label
1	1	1	+1
2	-1	1	-1
Object 3	-1	-1	-1
4	1	-1	-1

$$\mathbf{V=} \sum_{j=1}^{n=2} w_j x_j + w_0 = w_1 x_1 + w_2 x_2 + w_0 = 0{,}3\,.(-1) + 0.(-1) - 0 = -0{,}3$$

The prediction value y' $= f(V_1) = -1$, *so* The prediction is correct, y = y', it's Okay.

Object 4

The initial weights:

$w_1 = 0{,}3 \quad w_2 = 0 \quad w_0 = 0$

The learning rate: $\alpha = 0{,}1$

Input Data	x1	x2	Label
1	1	1	+1
2	-1	1	-1
3	-1	-1	-1
Object 4	1	-1	-1

$$\mathbf{V=} \sum_{j=1}^{n=2} w_j x_j + w_0 = w_1 x_1 + w_2 x_2 + w_0 = 0{,}3\,.(1) + 0.(-1) - 0 = +0{,}3$$

The prediction value y' $= f(V_1)\ is\ postive(+1\ )$

Since, the prediction is incorrect y ≠ y', so we are updating the weights (w_1, w_2, w_0).

$$w_0 = w_0 + 0{,}1\big(-1 - (+1)\big)x_0 = 0 - 0{,}2.1 = -0{,}2$$
$$w_1 = w_1 + 0{,}1\big(-1 - (+1)\big)x_1 = 0{,}3 - 0{,}2.1 = 0{,}1$$
$$w_2 = w_2 + 0{,}1\big(-1 - (+1)\big)x_2 = 0 - 0{,}2.(-1) = 0{,}2$$

Final Weights after the First Epoch

Object 1

The initial weights:

$w_1 = 0{,}1 \quad w_2 = 0{,}2 \quad w_0 = -0{,}2$

The learning rate: $\alpha = 0{,}1$

Input Data	x1	x2	Label
Object 1	1	1	+1
2	-1	1	-1
3	-1	-1	-1
4	1	-1	-1

$$\mathbf{V=} \sum_{j=1}^{n=2} w_j x_j + w_0 = w_1 x_1 + w_2 x_2 + w_0 = 0{,}1\,.(1) + 0{,}2\,.(1) - 0{,}2 = +0{,}1$$

V > 0, so the prediction label $y' = f(V)\ is\ positive\ (+1)$, , and we know that the Actual label is positive (+1)

So The prediction is correct, y = y’, it’s Okay.

Object 2

The initial weights:

$w_1 = 0{,}1 \quad w_2 = 0{,}2 \quad w_0 = -0{,}2$

The learning rate: $\alpha = 0{,}1$

Input Data	x1	x2	Label
1	1	1	+1
Object 2	-1	1	-1
3	-1	-1	-1
4	1	-1	-1

$$\mathbf{V=} \sum_{j=1}^{n=2} w_j x_j + w_0 = w_1 x_1 + w_2 x_2 + w_0 = 0{,}1\,.(-1) + 0{,}2\,.(1) - 0{,}2 = -0{,}1$$

V < 0, so the prediction label $y' = f(V)\ is\ negative\ (-1)$, , and we know that the Actual label is negative (-1)

So The prediction is correct, y = y’, it’s Okay.

Object 3

The initial weights:

$w_1 = 0{,}1 \quad w_2 = 0{,}2 \quad w_0 = -0{,}2$

The learning rate: $\alpha = 0{,}1$

Input Data	x1	x2	Label
1	1	1	+1
2	-1	1	-1
Object 3	-1	-1	-1
4	1	-1	-1

$$\mathbf{V=} \sum_{j=1}^{n=2} w_j x_j + w_0 = w_1 x_1 + w_2 x_2 + w_0 = 0{,}1\,.(-1) + 0{,}2\,.(-1) - 0{,}2 = -0{,}5$$

V < 0, so the prediction label $y' = f(V)\ is\ negative\ (-1)$, , and we know that the Actual label is negative (-1)

So The prediction is correct, y = y', it's Okay.

Object 4

The initial weights:

$w_1 = 0{,}1 \quad w_2 = 0{,}2 \quad w_0 = -0{,}2$

The learning rate: $\alpha = 0{,}1$

Input Data	x1	x2	Label
1	1	1	+1
2	-1	1	-1
3	-1	-1	-1
Object 4	1	-1	-1

$$\mathbf{V=} \sum_{j=1}^{n=2} w_j x_j + w_0 = w_1 x_1 + w_2 x_2 + w_0 = 0{,}1\,.\,(1) + 0{,}2\,.\,(-1) - 0{,}2 = -0{,}3$$

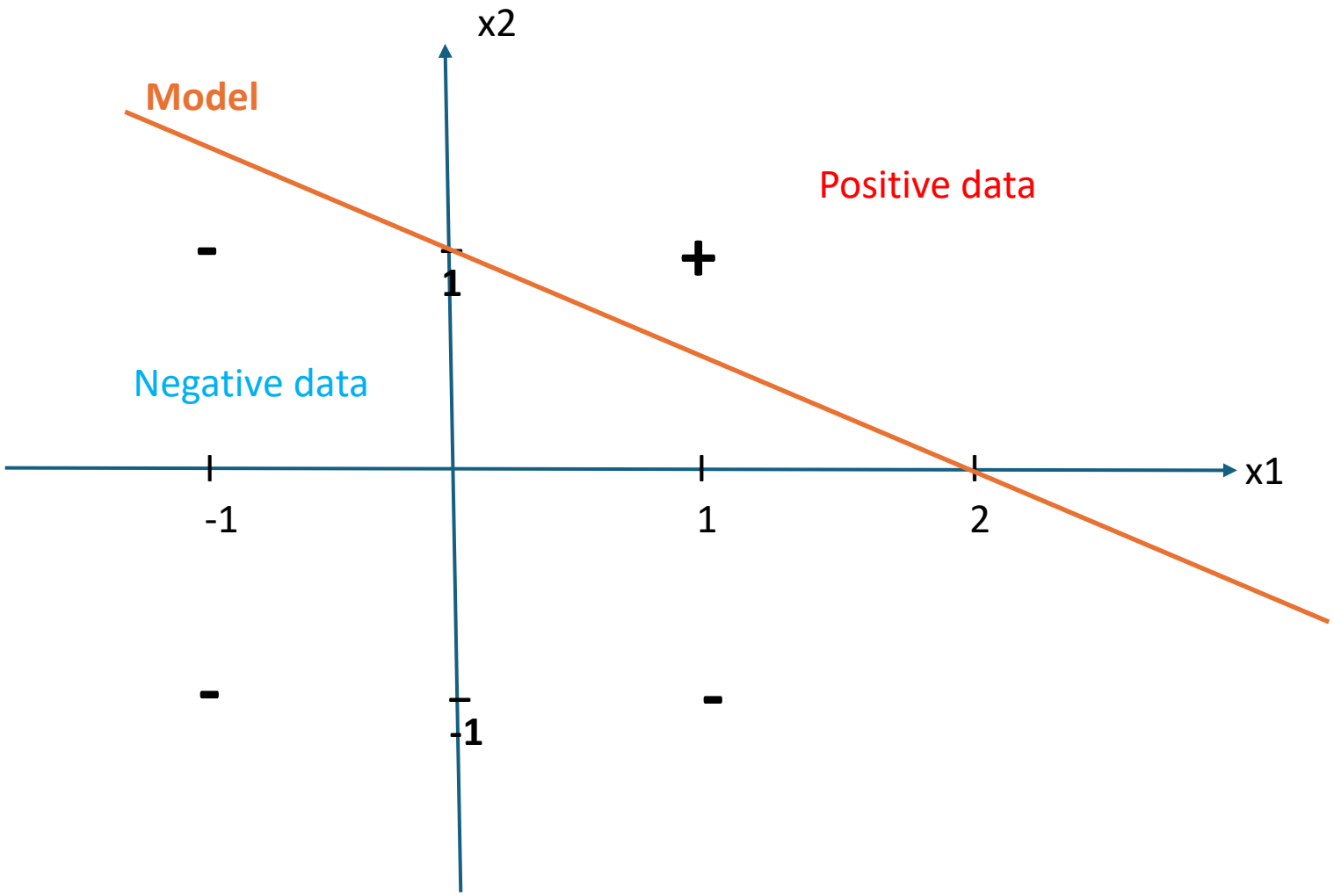
V < 0, so the prediction label $y' = f(V)\ is\ negative\ (-1)$, , and we know that the Actual label is negative (-1)

So The prediction is correct, y = y', it's Okay.

Final Weights after the second Epoch

The final weights:

$w_1 = 0{,}1 \quad w_2 = 0{,}2 \quad w_0 = -0{,}2$

The SLP model is :

$$\mathbf{0,1x_1 + 0,2\,x_2 - 0,2 = 0}$$